

ENSIL-ENSCI — Université de Limoges
Projet de fin d'études — MIX 5 — Promotion 2026

Projet V.E.S.P.A.

Tourelle optronique de suivi prédictif par stéréo-vision
à neutralisation laser dynamique — prototype mk.2

Difficultés techniques

Verrous levés et verrous reportés

Équipe

Antonin Delmas
Clément Desmedt
Clément Gilson

Encadrement

Michel Ousset — encadrant
Thierry Cortier — enseignant

Document de clôture — juillet 2026
Rédigé rétrospectivement à partir du code, des archives de conversation,
des fichiers de configuration du Jetson et des relevés de banc.

Table des matières

1	Comment lire ce document	2
2	Verrous levés	3
2.1	Faire cohabiter deux caméras MIPI à adresse I ² C identique	3
2.2	Synchroniser les deux capteurs à quelques microsecondes près	3
2.3	Les réglages optiques ne s'appliquaient pas	4
2.4	La console série se taisait dès que le potentiomètre était branché	4
2.5	Le SPI des encodeurs pouvait mourir sans raison	4
2.6	Deux programmes se disputaient le bus I ² C	5
2.7	<code>jetson-io.py</code> détruisait la configuration des caméras	5
2.8	Développer le protocole CAN avant d'avoir le bus	5
2.9	Les transceivers CAN étaient des contrefaçons	6
3	Verrous reportés	7
3.1	Le bus CAN physique : la liaison a fonctionné, puis elle a été perdue	7
3.2	Reconnaissance biologique : VespAI est en couleur, nos capteurs non	9
3.3	Le nœud de coordination, le watchdog, et le tir	10
3.4	La profondeur de champ létale : un résultat inattendu	11
3.5	Verrous mineurs et dette technique	12
4	Ce que l'assistance par IA a coûté, et ce qu'elle a rapporté	13
5	Synthèse : par où reprendre	15

1 Comment lire ce document

Ce document recense les obstacles **non triviaux** rencontrés au cours du projet V.E.S.P.A. — ceux dont la solution n'était ni évidente ni documentée, et ceux qui n'ont pas été résolus.

Il ne s'agit pas d'un journal de bord. Un bug de compilation n'y figure pas ; un bug qui a coûté une semaine, faussé une mesure ou contraint une décision d'architecture, oui.

Convention

LEVÉ — le verrou a été franchi. La cause racine et le correctif sont donnés, et le correctif est dans le dépôt.

REPORTÉ — le verrou est ouvert. L'état exact des tentatives est donné, ainsi que la piste jugée la plus prometteuse. **C'est là que doit commencer toute reprise du projet.**

À lire avant de reprendre le matériel

Plusieurs de ces verrous ne sont *pas visibles dans le code*. Ils sont dans le silicium, dans le brochage de la carte, ou dans un fichier de configuration du noyau. Un repreneur qui lirait uniquement les sources rencontrerait à nouveau, une par une, les mêmes impasses. C'est précisément la raison d'être de ce document.

Une remarque transverse, et elle est structurante. Ce projet a été développé avec une assistance forte de modèles de langage. Cette assistance a été décisive sur le volume de code produit. Mais **sur les verrous matériels, elle a été prise en défaut de manière répétée**, avec une grande assurance de façade. Les erreurs de diagnostic les plus coûteuses du projet sont des hypothèses d'IA plausibles, bien argumentées, et fausses — qui n'ont été écartées que par la mesure. La section 4 en fait le bilan explicite, car c'est une leçon de méthode réutilisable.

2 Verrous levés

2.1 Faire cohabiter deux caméras MIPI à adresse I²C identique

LEVÉ — Double flux OV9281 simultané

Symptôme. Les deux modules Arducam OV9281 portent la *même* adresse I²C, figée en dur dans le silicium du capteur. La documentation courante conclut qu'un multiplexeur I²C matériel est nécessaire pour en exploiter deux simultanément.

Cause racine. L'adresse est bien unique par capteur, mais les deux caméras ne partagent pas le même bus : chaque port CSI du Jetson dispose de son propre bus I²C. Le conflit d'adresse est donc *apparent*, pas réel — à condition que le noyau instancie bien un sous-périphérique par port.

Correctif. Noyau Arducam personnalisé + *overlay* d'arbre de périphériques dédié au mode double : `tegra234-p3767-camera-p3768-arducam-dual.dtbo`, chargé au démarrage (voir `jetson/extlinux.conf.reference`). Les deux capteurs apparaissent alors comme deux nœuds `/dev/video*` indépendants, chacun avec son propre sous-périphérique de contrôle. Aucun multiplexeur matériel n'est nécessaire.

C'est le résultat que le rapport de soutenance qualifiait d'« inédit », et la formulation est justifiée : la documentation constructeur comme les fils de support concluent à la nécessité d'un multiplexeur matériel. Le blocage est réputé réhibitoire ; il ne l'est pas. La solution économise un composant et une source de latence.

2.2 Synchroniser les deux capteurs à quelques microsecondes près

LEVÉ — Déclenchement matériel commun par PWM du Tegra

Symptôme. La triangulation stéréo est faussée dès que les deux images ne sont pas prises au même instant. Sur une cible mobile, un déphasage de quelques millisecondes suffit à déplacer le point 3D reconstruit de plusieurs centimètres. Or Linux n'est pas un RTOS : un déclenchement logiciel des deux caméras est soumis à l'ordonnanceur et dérive.

Cause racine. Toute synchronisation pilotée par le CPU est structurellement inadéquate. Il faut sortir la génération d'horloge du domaine logiciel.

Correctif. Les **deux** caméras sont forcées en mode esclave (`trigger_mode=1`) et reçoivent le *même* front de déclenchement, généré par le **PWM matériel du Tegra** — indépendant de la charge CPU :

```
/sys/class/pwm/pwmchip3/pwm0/period      = 16666666 ns -> 60 Hz
/sys/class/pwm/pwmchip3/pwm0/duty_cycle = 1666666 ns -> 10 %
```

La simultanéité est ainsi vraie *par construction* : les deux capteurs exposent sur le même front électrique. Implémentation : `HardwareSyncController.cpp` (`armTrigger`, `forceSlaveMode`).

Résultat mesuré. Dérive inter-caméras de **0,000 ms**. Un garde-fou logiciel subsiste dans `captureSynchronizedPair()` : toute paire dont les horodatages s'écartent de plus de 5 ms est rejetée et resynchronisée par consommation de la file la plus en retard.

Pourquoi ce résultat est réellement inédit. Ce n'est pas une figure de style. La position du constructeur est explicite : la synchronisation matérielle de plusieurs caméras passe par un accessoire propriétaire, le **Camarray HAT** (déclinaisons *Stereoscopic* et *Quadrascopic*), qui agrège les modules et leur injecte une horloge maîtresse par la nappe [1]. Les cartes UC-788 vendues dans ces kits subissent d'ailleurs des modifications d'usine irréversibles (oscillateur 24 MHz non peuplé, adresse I²C altérée au niveau des résistances) qui les rendent inutilisables seules. Sur une carte UC-788 *standard*, la piste qui relie la broche **XVS/FSIN** du capteur aux

pastilles de déclenchement est **physiquement coupée en sortie d'usine** — un emplacement 0402 laissé vide fait office de circuit ouvert.

Autrement dit : le chemin officiel pour obtenir un déclenchement stéréo matériel est d'**acheter la carte du constructeur**. Le *vision node* de VESPA ne l'a pas fait. Il obtient le même résultat sur **deux cartes UC-788 standard**, en prenant l'horloge maîtresse là où personne ne la cherche : le **générateur PWM matériel du Tegra lui-même**, câblé sur l'en-tête 40 broches. Aucun accessoire, aucun microcontrôleur intermédiaire, aucune dérive logicielle. **Le constructeur affirme que cela demande son matériel ; la mesure dit le contraire.**

2.3 Les réglages optiques ne s'appliquaient pas

LEVÉ — Fenêtre d'application des contrôles V4L2

Symptôme. Les réglages d'exposition et de gain écrits dans le JSON étaient silencieusement ignorés par le capteur. Aucune erreur, aucun code de retour : simplement, l'image ne changeait pas.

Cause racine — double.

1. **Fenêtre temporelle.** Les registres actifs du capteur ne « écoutent » pas tant que ses PLL ne sont pas stabilisées. Pousser les réglages avant que le flux ne tourne revient à écrire dans le vide.
2. **Nom de contrôle.** Le pilote V4L2 attend strictement `analogue_gain` (orthographe britannique). Toute variante (`gain`, `analog_gain`) échoue *en silence*. L'exposition fonctionnait, le gain non — ce qui rendait le diagnostic contre-intuitif.

Correctif. Séquence d'initialisation stricte dans `initializeRig()` : démarrer les flux → attendre 500 ms → charger le JSON → appliquer → forcer le mode esclave → purger les files. Le ticket de dette technique correspondant est **clos**.

2.4 La console série se taisait dès que le potentiomètre était branché

LEVÉ — Conflit PA2 — potentiomètre vs USART2_TX

Symptôme. Sur le `motion_node`, brancher le potentiomètre de réglage fin de l'axe PAN rendait la console série muette ou illisible. Aucun lien apparent entre les deux.

Cause racine. Le shield X-NUCLEO-IHM16M1 câble son entrée « SPEED » (potentiomètre) sur PA2. Mais sur la carte NUCLEO-G431RB, PA2 est **déjà câblée sur USART2_TX**, c'est-à-dire sur le port COM virtuel du ST-LINK. Y injecter une tension analogique revient à court-circuiter la sortie série.

Correctif. Potentiomètre PAN déplacé sur PB14 (`pinout.h`). Le câblage de référence est `docs/annexes/cablage.md` ; le manifeste de câblage d'origine, qui indiquait encore PA2, a été retiré du dépôt à la clôture du projet.

2.5 Le SPI des encodeurs pouvait mourir sans raison

LEVÉ — PA4 : NSS matériel vs retour de courant

Cause racine. PA4 est simultanément la broche `SENSEW` du moteur 1 (retour de courant phase W) et le **NSS matériel du SPI1**. Or SPI1 porte les deux encodeurs AS5048A, dont dépend tout l'asservissement. Un NSS matériel laissé flottant peut faire basculer le périphérique SPI en mode esclave et interrompre la lecture des encodeurs.

Correctif — un arbitrage, pas une solution. PA4 est maintenue haute et réservée au SPI. Le retour de courant de la phase W du moteur 1 est sacrifié. C'est acceptable ici : la FOC de SimpleFOC reconstruit la phase manquante ($i_W = -i_U - i_V$), et surtout

l'asservissement actuel est en position sur encodeur, sans boucle de courant. **Un repreneur qui voudrait activer la mesure de courant devra rouvrir cet arbitrage.**

2.6 Deux programmes se disputaient le bus I²C

LEVÉ — Verrou d'exclusion mutuelle matériel

Symptôme. Lancer un outil Python de calibration pendant que le nœud C++ tournait corrompait les réglages du capteur, voire figeait le flux.

Cause racine. Les deux processus écrivent sur les mêmes sous-périphériques I²C. Rien, au niveau du système, ne les en empêche.

Correctif. Verrou exclusif flock sur /tmp/vespa_hardware.lock, pris au démarrage de HardwareSyncController (acquireHardwareLock()). Le second processus **refuse de démarrer** avec un message explicite.

Ce n'est pas un bug : lancer le nœud de vision et un outil Python simultanément échoue *volontairement*.

2.7 jetson-io.py détruisait la configuration des caméras

LEVÉ — Abandon de l'outil NVIDIA au profit d'un arbre de périphériques maîtrisé

Symptôme. Impossible d'avoir *en même temps* le pilote caméra et le PWM matériel. Configurer l'un via jetson-io.py cassait l'autre.

Cause racine. jetson-io.py régénère et réapplique les *overlays* d'arbre de périphériques de façon globale : il écrase les configurations de multiplexage existantes au lieu de les compléter.

Correctif. L'outil est **proscrit**. La configuration matérielle est gérée par des *overlays* explicites, versionnés, chargés depuis /boot/extlinux/extlinux.conf. Les fichiers sont conservés dans jetson/ (voir docs/annexes/jetson_setup.md).

Ne pas lancer jetson-io.py

Sur une machine reconstruite selon ce dépôt, exécuter jetson-io.py et sauvegarder **cassera la synchronisation stéréo**. L'*overlay* PWM (jetson-io-hdr40-user-custom.dtbo, broches 32 et 33) doit être préservé tel quel.

2.8 Développer le protocole CAN avant d'avoir le bus

LEVÉ — Bus virtuel vcan0

Pour ne pas séquentialiser le projet derrière l'intégration matérielle, toute la couche protocolaire a d'abord été développée et validée sur une interface CAN **virtuelle** :

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan && sudo ip link set up vcan0
candump vcan0
```

Les trames TARGET_VEC (0x020, 7 octets) et FIRE_CMD (0x010) y ont été observées, correctement sérialisées, à la cadence attendue. vcan0 étant vu par SocketCAN exactement comme une interface réelle, le code applicatif n'a eu **aucune ligne à changer** lors du passage sur can0 physique. C'était le but, et cela a marché : la sérialisation, les identifiants et les priorités étaient justes du premier coup sur le cuivre.

2.9 Les transceivers CAN étaient des contrefaçons

LEVÉ — Clones 5 V vendus pour des SN65HVD230 3,3 V

Symptôme. Comportement du bus **asymétrique**, ce qui n'a aucun sens sur un bus différentiel. Relevé du noyau sur le Jetson :

```
$ ip -details -statistics link show can0
can state BUS-OFF (berr-counter tx 248 rx 0) restart-ms 100
bitrate 500000 sample-point 0.870
re-started bus-errors arbit-lost error-warn error-pass bus-off
20362      0          0          20363      20363      20363
RX: bytes  packets  errors  dropped
   772684   111909    0       1
TX: bytes  packets  errors  dropped
  121076   30458    0       26
```

Lecture du relevé. La réception est *parfaite* : **111 909 trames, zéro erreur**. L'émission, elle, est morte : le compteur TX est figé sur la valeur atteinte lors du dernier test en bouclage interne, le contrôleur part en BUS-OFF et se relance 20 362 fois. Le Jetson *entend* parfaitement le STM32 ; il n'arrive pas à lui *répondre*.

Cause racine. Les modules WCMCU-230, vendus comme portant un **SN65HVD230** de Texas Instruments (3,3 V), sont des **contrefaçons** : boîtiers **poncés puis re-marqués au laser**, abritant un clone qui fonctionne en réalité en **5 V**. D'où l'asymétrie : la sortie 5 V du module attaque sans peine l'entrée 3,3 V du Jetson (réception impeccable), alors que la sortie 3,3 V du Jetson n'atteint jamais le seuil logique haut de l'entrée 5 V du clone (émission jamais transmise sur le bus).

Correctif. Alimenter le module en **5 V** sur la broche sérigraphiée « 3V3 », puis ramener sa sortie CRX vers l'entrée 3,3 V du Jetson par un **pont diviseur résistif** 1 k Ω / 2 k Ω . La sortie CTX du Jetson (3,3 V) attaque directement le module sans adaptation.

Ne pas utiliser de convertisseur de niveau à MOSFET

Les petites cartes bidirectionnelles à MOSFET et résistances de tirage (celles prévues pour l'I²C) sont **trop lentes pour du CAN à 500 kbit/s** : les fronts deviennent des « ailerons de requin » que le contrôleur du Tegra classe en bruit et ignore. Un simple pont diviseur, lui, n'introduit quasiment aucune capacité et garde le front franc.

Validation. Le correctif a été qualifié sur un banc dédié à **deux ESP32** identiques avant d'être reporté sur le matériel du projet — on ne rebranche pas un montage douteux sur un Jetson à 300 €.

Ce verrou est le plus coûteux du projet en temps, et le seul qu'aucune quantité de relecture de code ne pouvait résoudre : **le défaut était dans le silicium, et le silicium mentait sur son propre marquage.**

3 Verrous reportés

3.1 Le bus CAN physique : la liaison a fonctionné, puis elle a été perdue

REPORTÉ — CAN0 du Jetson — le routage des broches ne survit pas au démarrage à froid

Le bus a bel et bien fonctionné sur cuivre : le Jetson a reçu **111 909 trames du STM32, sans une seule erreur** (§2.9). Ce qui n'a pas été refermé avant la fin du projet, c'est de rendre ce routage **persistant** — et de reconfirmer le sens Jetson → STM32 après la réparation des transceivers.

Ce qui est acquis, et qu'il ne faut pas refaire. Le relevé du noyau du 21 mars 2026 est sans ambiguïté : `can0` est bien l'interface du contrôleur matériel du Tegra (`parentdev c310000.mttcan`), à 500 kbit/s, et elle a encaissé **111 909 trames, 0 erreur de réception**, soit environ **une heure et demie** de trafic `POS_MOTION` continu à 20 Hz. Un tel compteur ne peut pas être produit par l'écho local de `SocketCAN` : **ces trames sont physiquement arrivées par le cuivre**. Il en découle, sans autre preuve à apporter, que le *pinmux*, le câblage, la terminaison et le *bit timing* étaient **corrects**.

Ce qui restait ouvert. Deux points, et deux seulement :

1. **Le sens Jetson → STM32 n'a jamais été confirmé de bout en bout** après la correction des transceivers contrefaits. Avant correction, il était en `BUS-OFF` — ce qui est *expliqué* par la contrefaçon, mais n'a pas été re-testé une fois le pont diviseur en place.
2. **L'overlay de *pinmux* a disparu de /boot.** Après le recâblage et un démarrage à froid, le Jetson est devenu sourd. Diagnostic final :

```
$ find /boot -name "*can*.dtbo"
$                                     # <- vide : le .dtbo n'est plus là

$ sudo bash -c "cat /sys/kernel/debug/pinctrl/*/pinmux-pins | grep -i paa"
pin 0 (CAN0_DOUT_PAA0): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 1 (CAN0_DIN_PAA1): (MUX UNCLAIMED) (GPIO UNCLAIMED)
```

Le contrôleur `mttcan` se charge toujours et `can0` existe toujours — mais ses broches ne sont plus *réclamées* : le silicium CAN n'est plus relié à l'en-tête 40 broches. Le nœud émet dans le vide. **Ce n'est pas une régression logicielle : c'est le binaire d'overlay qui n'est plus là où le *bootloader* va le chercher.**

Le piège de brochage. Sur le Tegra 234 :

Broches SoC	Contrôleur	En-tête 40 broches
PAA0 / PAA1	CAN0	broche 31 (TX) / broche 29 (RX)
PAA2 / PAA3	CAN1	—

Assigner la fonction `can0` aux broches **PAA2/PAA3** revient à demander au silicium de router *CAN0* sur les broches physiques de *CAN1*. C'est l'erreur classique sur cette carte, et elle ne produit aucun message d'erreur : simplement, plus rien ne sort.

L'overlay, retrouvé et remis en place. Le bon binaire a depuis été récupéré : il est de nouveau dans `/boot`, référencé par la ligne `OVERLAYS` de `/boot/extlinux/extlinux.conf` (modèle fourni : `jetson/extlinux.conf.reference`), et sa copie de référence est versionnée dans `jetson/dts/can0-pinmux.dtbo`, avec sa source `.dts`. Son en-tête interne annonce « `CAN0`

Hardware Override V5 » — cinquième itération. Il déclare les nœuds corrects (`can0_dout_paa0`, `can0_din_paa1`, fonction `can0`) et neutralise `paa2/paa3`. **C'est cette version qui a produit les 111 909 trames.**

Une hypothèse répandue — et fausse

La dernière piste explorée avant l'arrêt du projet attribuait la panne à un *overlay* `jetson-io-hdr40-user-custom.dtbo` « empoisonné », qui aurait contenu une mauvaise affectation CAN sur **PAA2/PAA3**.

Cette hypothèse est démentie par le fichier lui-même. Son inspection (juillet 2026) montre qu'il ne configure *que* deux broches : `hdr40-pin32 → soc_gpio19_pg6` et `hdr40-pin33 → soc_gpio21_ph0` — les broches du PWM de synchronisation. **Il ne contient aucune configuration CAN.**

Ne pas perdre de temps à « réparer » ce fichier : il n'est pas en cause, et le modifier casserait la synchronisation stéréo. La cause est ailleurs.

Procédure de reprise. Il ne s'agit pas d'une recherche, mais d'une remise en état. Dans l'ordre :

1. **Vérifier l'*overlay*** : `can0-pinmux.dtbo` doit être dans `/boot` et sur la ligne **OVERLAYS** de `extlinux.conf` — il y est en l'état. S'il a disparu (mise à jour JetPack), le recopier depuis `jetson/dts/` et redémarrer.
2. **Vérifier que les broches sont réclamées** — c'est le contrôle qui tranche :

```
sudo bash -c "cat /sys/kernel/debug/pinctrl/*/pinmux-pins | grep -i paa"
# ATTENDU : pin 0 (CAN0_DOUT_PAA0): c310000.mttcan <- reclamee
# ECHEC   : pin 0 (CAN0_DOUT_PAA0): (MUX UNCLAIMED) <- overlay non
        applique
```

3. **Remonter le montage transceiver** avec le correctif 5 V + pont diviseur (§2.9) — *et non* le câblage 3,3 V « théorique ».
4. **Confirmer les deux sens** : `candump can0` doit montrer les trames 030 du STM32 *et* le compteur TX du Jetson doit progresser sans partir en **BUS-OFF**.

Le piège qui a coûté le bus : une mise à jour efface /boot

Une mise à jour de JetPack, ou toute opération qui régénère `/boot`, peut **supprimer silencieusement** le `.dtbo` sans toucher à la ligne **OVERLAYS** qui le référence. Le *bootloader* ne trouve pas le fichier, **n'émet aucune erreur**, et démarre sans l'*overlay*. Le système est alors *exactement* identique à un système sain — sauf que le CAN est muet.

Après **toute** mise à jour du Jetson, vérifier : `find /boot -name "*can*.dtbo"`. Si la commande ne renvoie rien, le bus est mort et c'est la seule chose à réparer.

Repli, si le *pinmux* résiste. Un contrôleur CAN externe sur **SPI** (MCP2515 ou équivalent) donne un bus fonctionnel sans dépendre du *pinmux* du Tegra ni de son arbre de périphériques. On perd le CAN natif; on gagne l'immunité aux mises à jour de JetPack. **Sous contrainte de temps, c'est l'option raisonnable.**

Ce que ce verrou n'est pas. Il n'est ni logiciel, ni protocolaire, ni un défaut de conception du bus. Le protocole a tourné sur le cuivre. **Seule la persistance du routage des broches est en cause.**

3.2 Reconnaissance biologique : VespAI est en couleur, nos capteurs non

REPORTÉ — Incompatibilité VespAI / capteur monochrome

Le système ne sait pas reconnaître un frelon. Il suit un marqueur ArUco.

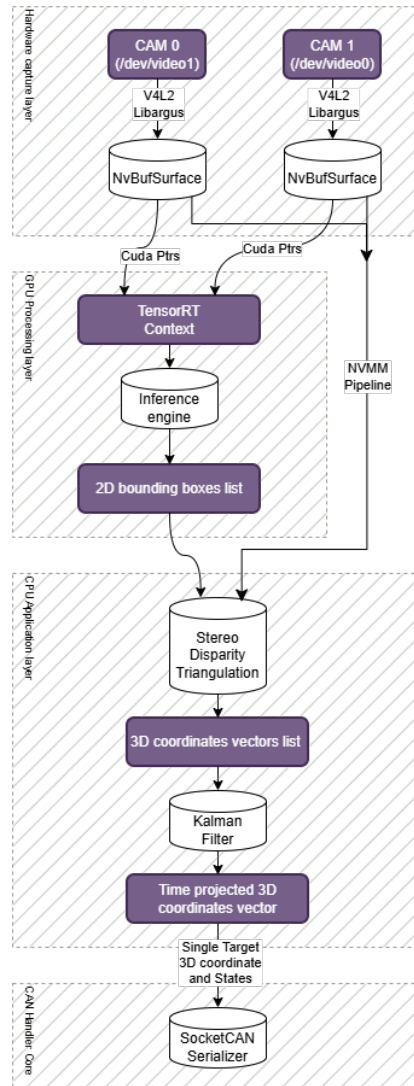


FIGURE 1 – Le pipeline de vision tel qu’il était prévu — et tel qu’il n’a pas été livré. Le bloc *TensorRT Context / Inference engine* devait porter la détection biologique sur GPU. C’est précisément ce bloc qui est tombé : il est remplacé, dans le système livré, par une détection de **marqueur ArUco** sur CPU (OpenCV). Tout le reste de la chaîne — triangulation stéréo, filtre de Kalman, sérialisation SocketCAN — est en revanche opérationnel et conforme à ce schéma. Le nœud de vision ne dépend donc aujourd’hui **ni de CUDA, ni de TensorRT, ni de VPI**.

L’enchaînement des contraintes. Ce verrou n’est pas une erreur : c’est une cascade logique dont chaque maillon est justifié, et dont le résultat est une impasse.

1. Le flou de mouvement détruit la détection → il faut un **obturateur global**.

2. Aucun capteur **trichrome** à obturateur global n'existe à un prix accessible → le capteur sera **monochrome** (OV9281).
3. Le modèle VespAI [3] est entraîné sur des images **RGB** → **incompatible**.

Ce qui a été tenté. Une « chirurgie réseau » sur les poids pré-entraînés, sans réentraînement. L'idée est mathématiquement exacte : si l'on duplique le canal gris sur les trois canaux RGB, la première convolution calcule

$$\text{sortie} = I_R W_R + I_G W_G + I_B W_B = I_{\text{gris}} (W_R + W_G + W_B),$$

donc **sommer les poids de la première couche sur la dimension canal** produit un modèle à 1 canal *strictement équivalent*, sans perte, sans jeu de données, sans entraînement.

```
new_weights = torch.sum(old_weights, dim=1, keepdim=True) # (out,3,k,k) -> (out,1,k,k)
first_conv_layer.weight = nn.Parameter(new_weights)
first_conv_layer.in_channels = 1
```

Pourquoi cela a échoué quand même. La transformation est exacte, et le modèle obtenu tourne vite (compilé en TensorRT FP16, entrée native $1,280 \times 800$ — les deux dimensions étant multiples de 32, aucun redimensionnement n'est requis, donc pas de recours au VIC). **Mais l'équivalence mathématique ne crée pas d'information.** Le modèle reste entraîné sur des statistiques de couleur : il détecte des frelons *imprimés sur papier blanc*, et échoue en conditions réelles, avec des faux positifs massifs. Surtout, la distinction *V. velutina* / *V. crabro* — l'exigence éthique centrale du projet — repose largement sur la **coloration** [4] : elle est **structurellement impossible** en monochrome à partir d'un modèle couleur.

Décision. À trois semaines de la soutenance, le réentraînement complet (estimé 1,5 à 2,5 semaines, sans garantie de résultat, le jeu de données n'étant pas public) a été jugé trop risqué. **Repli assumé sur un marqueur ArUco** [2] : haut contraste, natif en niveaux de gris, aucun entraînement, pose 3D exacte. Le marqueur **tient lieu de cible** et permet de valider toute la chaîne aval (triangulation, prédiction, asservissement) — qui est, elle, indépendante de la nature de la cible.

Pour la reprise

La voie n'est pas la « chirurgie réseau ». C'est la **constitution d'un jeu de données monochrome** (images issues de l'OV9281 lui-même, en conditions réelles) et le réentraînement d'un détecteur dessus. Tant que la discrimination inter-espèces ne sera pas rétablie, **le système ne doit pas tirer** : il ne sait pas distinguer une espèce invasive d'une espèce protégée.

3.3 Le nœud de coordination, le watchdog, et le tir

REPORTÉ — Watchdog non implémenté — laser jamais mis sous tension

Le `coordination_node` (ESP32) **n'existe pas**. Il est spécifié — ses trames CAN sont définies (`FIRE_CMD`, `POS_COORD`, `HB_COORD`) — mais aucune ligne de code ne l'implémente.

Or ce nœud porte trois fonctions, dont une est une condition de sécurité :

Fonction	État
Collimation dynamique	Loi de commande établie, mécanisme spécifié, jamais assemblé
Séquence de tir	Non implémentée
Watchdog	Non implémenté — <i>condition sine qua non du tir</i>

Conséquence, et elle est délibérée. Le watchdog est le mécanisme qui surveille les *heartbeats* de tous les nœuds et coupe l'alimentation générale en cas de défaut ou de silence. Sans lui, **rien ne garantit l'extinction du laser si un nœud se fige**.

La diode laser n'a jamais été alimentée

Ce n'est pas un retard : c'est un **refus**. Mettre sous tension une diode de **Classe 4** (voir docs/annexes/securite_laser.md) sans *interlock* fonctionnel aurait été une faute. Aucun tir n'a été effectué, et **aucun ne doit l'être** tant que le watchdog n'est pas opérationnel et testé.

Note de conception. Le watchdog est prévu *dans le nœud de tir* — c'est-à-dire au plus près de l'organe dangereux, sur un cœur CPU dédié. C'est le bon choix : un superviseur qui dépendrait du réseau pour couper le courant serait vulnérable à la panne même qu'il doit traiter. La coupure est **matérielle** (alimentation générale), l'inhibition logicielle du tir n'intervenant que pendant le temps de décharge des condensateurs.

3.4 La profondeur de champ létale : un résultat inattendu

REPORTÉ — Collimation dynamique — jamais assemblée ni étalonnée

Le mécanisme de collimation dynamique vise à **dégrader volontairement** le faisceau partout ailleurs qu'au point d'impact, afin de réduire la dangerosité intrinsèque du dispositif. Deux lentilles : **L1** (fournie avec le module, rapprochée de la source pour la faire *diverger*) et **L2** (longue focale, mobile, qui refocalise à une distance commandée).

La loi de commande a été établie analytiquement par les relations de conjugaison de Descartes, en éliminant la focale inconnue f_1 au profit de grandeurs mesurables (demi-divergence θ , diamètre D_{L2} du faisceau sur la lentille mobile) :

$$x_{\text{cible}} = x_{L2} + \frac{f_2 \cdot \frac{D_{L2}}{2 \tan \theta}}{\frac{D_{L2}}{2 \tan \theta} - f_2}$$

Cette relation, quadratique, est approximée par un polynôme à 10^{-4} mm près — plus rapide à évaluer sur microcontrôleur.

Le résultat qu'il faut retenir. Le tracé de densité de puissance (simulation/optique/Laser_Optics_v1.m) donne deux enseignements **opposés**, et c'est le second qui compte :

- **Bonne nouvelle.** Le seuil d'embrasement de matières sèches n'est franchi que sur ≈ 50 mm de part et d'autre de la cible. Un tir manqué venant frapper le sol présente donc un risque d'incendie **limité**. L'objectif du mécanisme est atteint.
- **Mauvaise nouvelle, et elle domine.** Le seuil de dangerosité **oculaire** reste dépassé sur près de **200 mètres**. La collimation dynamique **ne rend pas le dispositif sûr** : elle réduit le risque d'incendie, pas le risque oculaire.

Avertissement

Il serait dangereux de conclure de ce mécanisme que le système est « moins dangereux ». Il ne l'est pas pour l'œil. La collimation dynamique est une mesure de mitigation **partielle**, en aucun cas un substitut aux protections (lunettes, *interlock*, confinement).

3.5 Verrous mineurs et dette technique

Sujet	Description	État
Appels <code>std::system</code>	Le pilotage du PWM passe par des appels shell (<code>echo > /sys/class/pwm/...</code>). Coûteux et fragile. À remplacer par des écritures <code>sysfs</code> directes en C++.	Ouvert
Chemins absolus	Les <code>#define</code> de scripts PWM sont des chemins absolus : déplacer le dépôt casse la compilation. À rendre relatifs ou configurables.	Ouvert
Échelle d'exposition	L'OV9281 n'utilise pas le même calcul de temps-ligne en mode maître et en mode déclenchement externe. L'outil <code>app_EG.py</code> calibre donc dans un régime différent de celui du <i>pipeline</i> C++ : les valeurs ne correspondent pas 1 :1.	Ouvert
Timing des contrôles V4L2	Attendre que le flux tourne avant de pousser les réglages.	Clos
Suivi multi-cibles	Un seul filtre de Kalman. Une nuée de frelons n'est pas gérée : ni association de données, ni priorisation.	Ouvert

4 Ce que l'assistance par IA a coûté, et ce qu'elle a rapporté

Cette section est inhabituelle dans un document technique. Elle est ici parce que le projet a été, de bout en bout, co-développé avec des modèles de langage, et que **le bilan n'est pas uniforme**. Un repreneur qui adopterait la même méthode doit savoir où elle porte et où elle nuit.

Ce qui a très bien fonctionné. La production de code. Le volume de logiciel écrit — chaîne V4L2 zéro-copie, sérialisation CAN, FOC, triangulation, filtre de Kalman, outils web de calibration — est hors de portée de trois étudiants sur la durée impartie sans cette assistance. Le rapport de soutenance le dit, et c'est exact.

Ce qui a coûté cher. Le diagnostic matériel. Les modèles produisent des hypothèses plausibles, argumentées, confiantes — et fausses. Elles sont d'autant plus coûteuses qu'elles sont *crédibles* : elles envoient chercher au mauvais endroit, parfois pendant des jours.

Affirmation de l'IA	Réalité établie par la mesure
« Deux OV9281 exigent un multiplexeur I ² C matériel »	Faux. Chaque port CSI a son propre bus. Un <i>overlay</i> adapté suffit.
« <code>jetson-io.py</code> : sélectionnez <code>can0</code> , cela routera les broches »	Faux, et destructeur : l'outil écrase la configuration caméra/PWM.
« L' <i>overlay</i> <code>jetson-io-hdr40-user-custom.dtbo</code> est empoisonné : il assigne CAN0 à PAA2/PAA3 »	Faux. Inspection du binaire : il ne contient <i>que</i> les deux broches PWM (32 et 33). Aucune configuration CAN. Le projet s'est arrêté sur cette fausse piste.
« Le bus CAN est muet : l'autre nœud est en mode <i>listen-only</i> / votre convertisseur de niveau est trop lent / votre code d'init est faux »	Toutes fausses. Le défaut était dans le <i>silicium</i> : des transceivers contrefaits (§2.9). Aucune hypothèse logicielle ne pouvait y mener. La cause a été trouvée en regardant physiquement la puce — poncée, re-marquée.
« La chirurgie réseau donnera un détecteur équivalent »	Mathématiquement exact, opérationnellement faux : l'équivalence formelle ne compense pas l'absence d'information couleur.

La règle qui s'en dégage. Sur ce projet, **toutes** les avancées matérielles décisives sont venues d'une *mesure* qui contredisait une hypothèse — pas d'un raisonnement. Le conflit PA2/série a été trouvé en débranchant. La contrefaçon des transceivers a été trouvée **à l'œil, sur la puce** — après que le modèle eut proposé, avec assurance, quatre causes logicielles successives toutes fausses. La fausse piste finale sur l'*overlay* aurait été écartée en *ouvrant le fichier incriminé* — ce qui prend deux minutes, et n'a pas été fait.

Le contre-exemple est instructif : c'est la seule fois où le relevé brut a été lu *pour ce qu'il disait* — une réception parfaite face à une émission morte — que la cause est devenue évidente. **Un bus différentiel ne peut pas être asymétrique. C'est l'anomalie qui portait la réponse**, et elle était visible dès le premier `ip -s link show can0`.

Recommandation au repreneur

Utilisez ces outils pour écrire du code : le gain est réel et considérable. Mais devant un symptôme matériel, **n'acceptez aucune cause racine qui n'ait été confirmée par une mesure**. Le coût d'une vérification est presque toujours inférieur au coût d'une fausse

piste bien argumentée.

5 Synthèse : par où reprendre

#	Chantier	Pourquoi dans cet ordre
1	Remettre le CAN physique en service	Le bus a déjà fonctionné (111 909 trames, 0 erreur) et l' <i>overlay</i> <code>can0-pinmux.dtbo</code> est en place dans <code>/boot</code> . Il ne reste qu'à remonter les transceivers avec le correctif 5 V et à reconfirmer le sens émission. C'est une remise en état, pas une recherche.
2	Implémenter le watchdog	Condition <i>sine qua non</i> de toute mise sous tension du laser de Classe 4. Aucun essai de tir avant cela.
3	Jeu de données monochrome + réentraînement	Rend au système sa fonction biologique et la discrimination inter-espèces. Sans cela, le système ne doit pas tirer.
4	Assembler et étalonner la collimation	Loi de commande établie ; mécanisme jamais monté. Attention : ne réduit <i>pas</i> le risque oculaire.
5	Gestion multi-cibles	Un filtre de Kalman par cible + association de données.

Le mot de la fin

Le système, tel qu'il est laissé, **suit** une cible en trois dimensions, à 60 Hz, avec une prédiction qui compense sa propre latence, et il oriente deux axes asservis en boucle fermée pour la garder en visée. Ce qui manque n'est pas le cœur du problème : c'est **un bus qui ne s'allume pas, un jeu de données qui n'existe pas, et un garde-fou qu'il fallait écrire avant d'oser tirer.**

Les fondations sont saines. Les trois chantiers sont indépendants. Bonne reprise.

Références

- [1] Arducam. UC-788 rev.B — OV9281 MIPI camera module carrier board. Datasheet, Arducam, 2023. Carte porteuse assurant la compatibilité de l’OV9281 avec l’écosystème NVIDIA Jetson. Source déterminante sur le déclenchement externe : le constructeur réserve la synchronisation matérielle multi-caméras à son accessoire propriétaire (*Camarray HAT*, déclinaisons Stereoscopic et Quadrascopic), et la piste reliant la broche XVS/FSIN du capteur aux pastilles de déclenchement est coupée en sortie d’usine (emplacement 0402 non peuplé) sur les cartes vendues seules. Le projet obtient néanmoins un déclenchement stéréo matériel sur deux cartes standard, en prenant l’horloge maîtresse sur le PWM matériel du Tegra. Voir l’étude dédiée *Etude_IA_Arducam_UC788_Trigger_Externe.pdf*.
- [2] Sergio Garrido-Jurado, Rafael Muñoz-Salinas, Francisco J. Madrid-Cuevas, and Manuel J. Marín-Jiménez. Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6) :2280–2292, 2014. Marqueurs ArUco. Solution de repli effectivement déployée en remplacement de VespAI : haut contraste, nativement adaptée aux capteurs monochromes, aucun entraînement requis.
- [3] Thomas A. O’Shea-Wheller, Andrew Corbett, Juliet L. Osborne, Mario Recker, and Peter J. Kennedy. VespAI : a deep learning-based system for the detection of invasive hornets. *Communications Biology*, 7(1) :354, 2024. Modèle de détection initialement retenu pour le projet. Architecture YOLOv5s + ResNet, score précision-rappel $\geq 0,99$. Entraîné sur images RGB : c’est cette contrainte qui a provoqué le verrou décrit au chapitre « Reconnaissance biologique ».
- [4] Tamás Sipos, Balázs Kolics, Éva Kolics-Horváth, Tamás Donkó, Ádám Csóka, Kristóf Kozma-Bognár, András Kovács, Sándor Farkas, Katalin Somfalvi-Tóth, and Sándor Keszthelyi. Comparative morphological analysis of yellow-legged hornet (*Vespa velutina nigrithorax*) and european hornet (*Vespa crabro*) based on modern imaging techniques. *Frontiers in Ecology and Evolution*, 13 :1624744, 2025. Source des paramètres morphologiques et des performances de vol (vitesse, manœuvrabilité, vol stationnaire) ayant servi au dimensionnement cinématique de la tourelle et au choix du taux de rafraîchissement. Établit également que *V. velutina* surpasse *V. crabro* en vol, et que les deux espèces se distinguent par la coloration — donc mal séparables en monochrome.